



Ch09

- 01. 참조
- 02. 포인터
- 03. 배열과 포인터
- 04. 메모리



배열명은?

- 포인터 상수이다.

```
int i ; // i는 int 변수
```

```
int* ip; // ip는 int 포인터 변수
```

```
int j;
```

```
ip = &j; // 포인터 변수이므로 값이 바뀔수 있다.
```

```
int A[5]; // A는 int 포인터 상수. 0번째 요소의 주소값을 갖는다.
```

```
ip = A; // 또는 ip = &A[0];
```

```
A = ip; // 불가. LHS가 상수이므로.
```



배열의 원소를 가리키는 포인터

- 포인터 변수를 사용해서 배열의 원소를 가리키는 예

```
int arrays[10];  
int* p = &arrays[5];
```

[그림 12-1]

포인터 변수에 정수를 더하기

- i 번째 원소를 가리키는 포인터 변수에 정수 n 을 더하면 포인터 변수는 $i + n$ 번째 원소를 가리키게 된다.

```
int array[10];

// 포인터가 array[5]를 가리키도록 만든다.
//int* p = &array[5];
int* p = (array + 5);
cout << "p = " << p << "\n";
cout << "&array[5] = " << &array[5] << "\n";

// 포인터가 array[6]을 가리키도록 만든다.
p++;

cout << "p = " << p << "\n";
cout << "&array[6] = " << &array[6] << "\n";
```

[그림 12-3]

- 실행 결과

[그림 12-2]

포인터 변수간의 뺄셈

- i 번째 원소를 가리키는 포인터 변수에서 j 번째 원소를 가리키는 포인터 변수를 빼면 결과는 i-j 가 된다.

pointer variable 간의 덧셈은 불가능하다 !

ex) ip = ip + cp;

```
short sArrays[10];  
//short* ps1 = &sArrays[3];  
//short* ps2 = &sArrays[7];  
// 결과를 출력한다.  
cout << "ps1 = " << ps1 << "\n";  
cout << "ps2 = " << ps2 << "\n";  
cout << "ps2 - ps1 = " << ps2 - ps1 << "\n";
```

short* ps1 = (sArrays + 3);

short* ps2 = (sArrays + 7);

[그림 12-6]

- 실행 결과

[그림 12-5]



포인터를 사용한 원소의 탐색

- 일반적인 방법을 사용한 원소의 탐색

```
int nArray[10];  
  
// 배열을 탐색하면서 값을 넣는다.  
for (int i = 0; i < 10; ++i)  
    nArray[i] = i;
```

- 포인터를 사용한 원소의 탐색

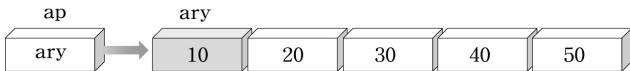
```
int nArray[10];  
int* p = &nArray[0]; // int* p = nArray;  
  
// 배열을 탐색하면서 값을 넣는다.  
for (int i = 0; i < 10; ++i)  
    *(p + i) = i;
```

▶ 포인터를 사용한 배열요소의 참조 정리

- 포인터(변수)로 배열요소를 참조하는 방법은 다음과 같다.

- ① 배열명을 사용한 index표현
- ② 배열명을 사용한 포인터표현
- ③ 배열명을 저장한 포인터변수를 사용한 포인터표현
- ④ 배열명을 저장한 포인터변수를 사용한 index표현

```
int ary[5]={10,20,30,40,50};  
int ary[5]={10, 20, 30, 40, 50};  
int *ap=ary;  
int* ap=ary; 또는 int* ap = &ary[0];
```



- | | | | | | |
|---|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|
| ① | <code>ary[0]</code> | <code>ary[1]</code> | <code>ary[2]</code> | <code>ary[3]</code> | <code>ary[4]</code> |
| ② | <code>*(ary+0)</code> | <code>*(ary+1)</code> | <code>*(ary+2)</code> | <code>*(ary+3)</code> | <code>*(ary+4)</code> |
| ③ | <code>*(ap+0)</code> | <code>*(ap+1)</code> | <code>*(ap+2)</code> | <code>*(ap+3)</code> | <code>*(ap+4)</code> |
| ④ | <code>ap[0]</code> | <code>ap[1]</code> | <code>ap[2]</code> | <code>ap[3]</code> | <code>ap[4]</code> |

→ 4가지 모두 첫번째 배열요소를 참조하는 표현식이다.



배열의 이름

- 배열의 이름은 첫번째 원소의 주소가 된다.

```
float f[5];  
if ( f == &f[0] )  
{  
    // 항상 이곳이 실행된다.  
}
```

- 배열의 이름을 사용해서 배열을 탐색하는 예

```
int nArray[10];  
  
// 배열을 탐색하면서 값을 넣는다.  
for (int i = 0; i < 10; ++i)  
    *(nArray + i) = i;
```